

# Internship Daily Report — Day 33

**Date:** March 18, 2026 **Domain:** Cloud / DevOps (AWS) **Focus Area:** AWS CloudFormation (Infrastructure as Code)

---

## Tasks Completed Today

### Task 1 — VPC + EC2 + RDS CloudFormation Template

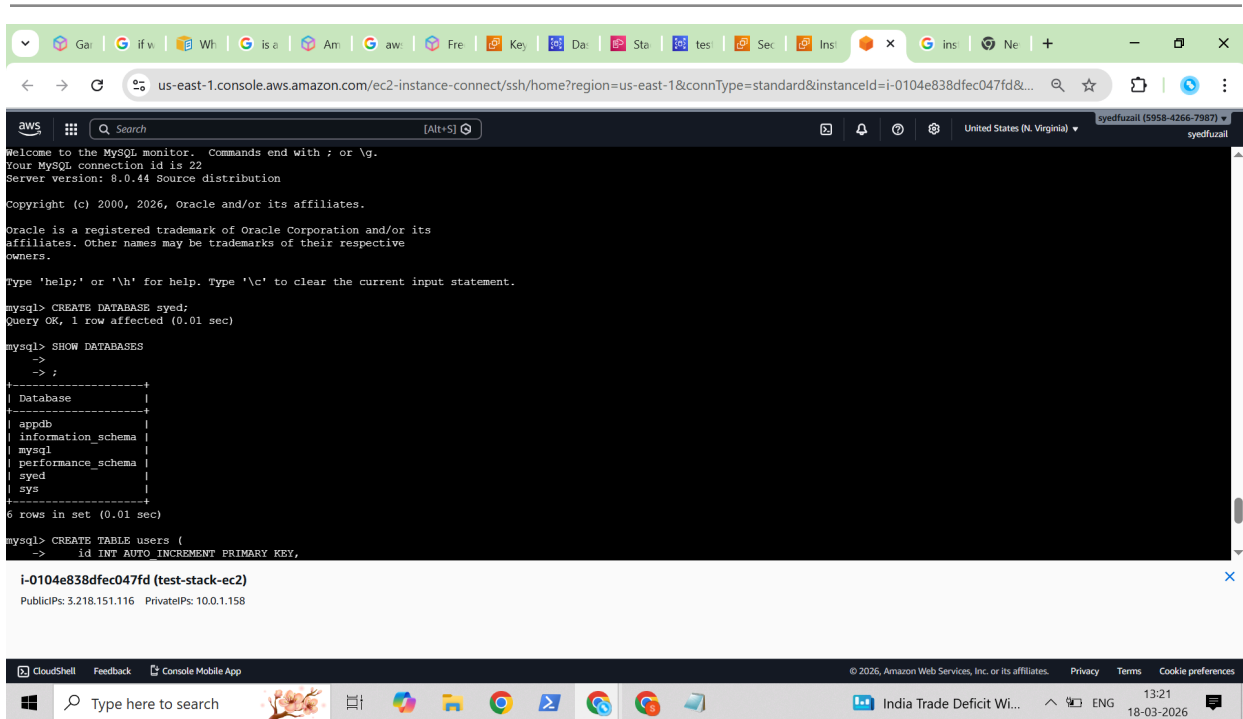
Built a complete AWS CloudFormation (CFT) template in YAML format to provision a full working cloud environment from scratch. The template covered:

- **VPC** with DNS hostnames and DNS support enabled (/16 CIDR)
- **Public subnets** (×2, different AZs) for EC2
- **Private subnets** (×2, different AZs) for RDS
- **Internet Gateway** attached to the VPC
- **NAT Gateway** (with Elastic IP) in the public subnet for outbound access from private subnets
- **Public & Private Route Tables** with correct associations
- **Security Groups:**
  - EC2 SG: SSH (port 22) from a specific IP, HTTP (80) and HTTPS (443) from anywhere
  - RDS SG: MySQL (3306) or PostgreSQL (5432) from the EC2 SG only (no CIDR — source SG reference)
- **EC2 Instance** on Amazon Linux 2 with Apache pre-installed via **UserData**
- **RDS Instance** (MySQL or PostgreSQL, selectable via parameter) in private subnets
- **Parameters** for: **KeyName**, **DBEngine**, **DBName**, **MasterUsername**, **MasterUserPassword** (**NoEcho: true**), **DBInstanceClass**, **EnableDeletionProtection**
- **Outputs** for: VPC ID, EC2 Public IP, EC2 Public DNS, RDS Endpoint, Security Group IDs, SSH command, DB connection hint

#### Key design decisions applied:

- Used `!Select [0/1, !GetAZs ""]` for dynamic AZ selection (region-agnostic)
- Used **DeletionPolicy: Snapshot** on RDS to preserve data on stack deletion
- Used **CreationPolicy** with `cfn-signal` on EC2 to gate stack completion until bootstrapping finishes

- RDS **BackupRetentionPeriod** set to 0 for free-tier compatibility (free tier does not support automated backups)
- **PreferredBackupWindow** removed when retention is 0 (it is only valid when backups are enabled)



The screenshot shows an AWS CloudShell terminal window with a dark background. The terminal displays the following text:

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22
Server version: 8.0.44 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE syed;
Query OK, 1 row affected (0.01 sec)

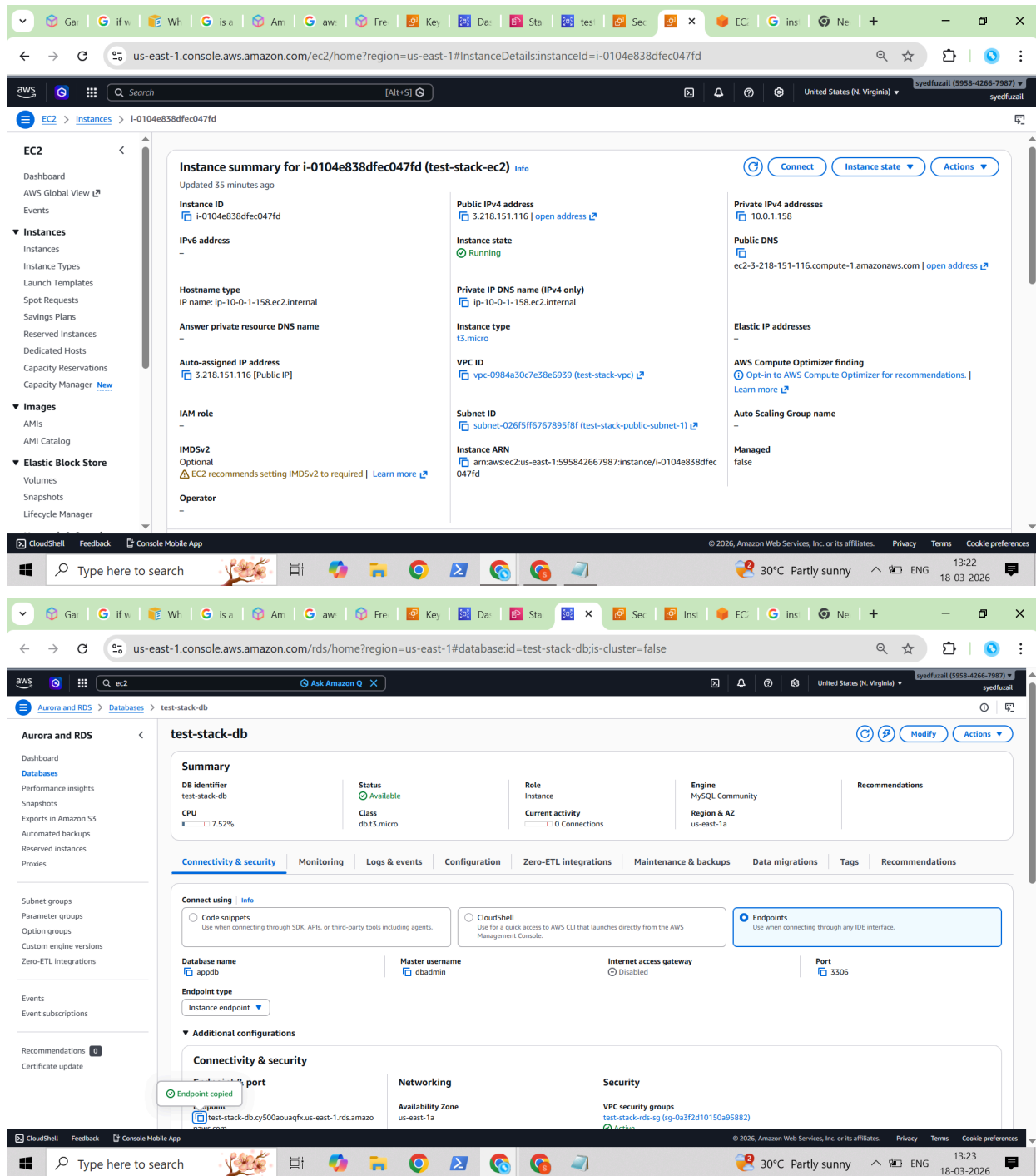
mysql> SHOW DATABASES
+-----+
| Database |
+-----+
| appdb    |
| information_schema |
| mysql    |
| performance_schema |
| syed     |
| sys      |
+-----+
6 rows in set (0.01 sec)

mysql> CREATE TABLE users (
  id INT AUTO INCREMENT PRIMARY KEY,
```

Below the terminal output, a blue box displays the instance ID and IP addresses:

```
i-0104e838dfec047fd (test-stack-ec2)
PublicIPs: 3.218.151.116  PrivateIPs: 10.0.1.158
```

The bottom of the screenshot shows the Windows taskbar with the search bar, task view button, and several application icons. The system tray on the right shows the date and time as 13:21 on 18-03-2026.



## Task 2 — Multi-OS EC2 CloudFormation Template (Windows + Linux)

Built a second CFT template that lets the user choose which OS to launch at deploy time — Linux only, Windows only, or both simultaneously.

**Parameter:**

OSChoice:

Type: String

AllowedValues:

- Linux
- Windows
- Both

### Conditions used:

Conditions:

LaunchLinux:

!Or

- !Equals [!Ref OSChoice, "Linux"]
- !Equals [!Ref OSChoice, "Both"]

LaunchWindows:

!Or

- !Equals [!Ref OSChoice, "Windows"]
- !Equals [!Ref OSChoice, "Both"]

Each resource (Security Group, EC2 Instance) and each Output carries a **Condition** key so only the selected OS resources are created. Shared infrastructure (VPC, IGW, subnet, route table) has no condition and is always created.

### Linux instance (Amazon Linux 2 + Apache):

- **UserData** bash script installs and starts Apache via **yum**
- Custom HTML landing page written to **/var/www/html/index.html**
- **cfn-signal** used to notify CloudFormation on successful boot
- **CreationPolicy** timeout: 10 minutes

### Windows instance (Windows Server 2022 + IIS):

- **UserData** PowerShell script installs IIS via **Install-WindowsFeature**
- Custom HTML landing page written to **C:\inetpub\wwwroot\index.html**
- Default IIS welcome page (**iisstart.htm**) removed so custom page loads first
- Windows Firewall rules opened for ports 80 and 443
- **cfn-signal.exe** used to notify CloudFormation on successful boot
- **CreationPolicy** timeout: 20 minutes (Windows bootstrapping takes longer)

### Security Groups:

| Instance | Inbound Rules                                                                               |
|----------|---------------------------------------------------------------------------------------------|
| Linux    | SSH (22) from allowed CIDR, HTTP (80) + HTTPS (443) from <code>0.0.0.0/0</code>             |
| Windows  | RDP (3389) from allowed CIDR, HTTP (80) + HTTPS (443) from <code>0.0.0.0/0</code> ,<br>ICMP |

**Public IP assignment:** handled at the subnet level via `MapPublicIpOnLaunch: true` — no instance-level property needed.

---

## Errors Faced & Fixed

### Error 1 — RDS Backup Retention (Free Tier Limit)

#### Error message:

The specified backup retention period exceeds the maximum available to free tier customers.

**Root cause:** `BackupRetentionPeriod: 7` is not allowed on free-tier AWS accounts.

#### Fix:

```
# Before
BackupRetentionPeriod: 7
PreferredBackupWindow: "03:00-04:00"

# After
BackupRetentionPeriod: 0
# PreferredBackupWindow removed — only valid when retention > 0
```

---

### Error 2 — Invalid EC2 Property: `AssociatePublicIpAddress`

#### Error message:

Properties validation failed for resource LinuxInstance with message: extraneous key [`AssociatePublicIpAddress`] is not permitted

**Root cause:** `AssociatePublicIpAddress` is **not** a valid top-level property on `AWS::EC2::Instance`. It is only valid inside a `NetworkInterfaces` block.

**Fix:** Removed the property entirely. Since the subnet already has `MapPublicIpOnLaunch: true`, public IPs are assigned automatically — no instance-level override is needed.

# WRONG — top-level, causes validation failure

Properties:

SubnetId: !Ref PublicSubnet

SecurityGroupIds:

- !Ref MySecurityGroup

AssociatePublicIpAddress: true # ❌ not valid here

# CORRECT — via subnet (preferred approach)

PublicSubnet:

Type: AWS::EC2::Subnet

Properties:

MapPublicIpOnLaunch: true # ✅ all instances get public IP automatically

# CORRECT — via NetworkInterfaces block (when explicit NIC config is needed)

Properties:

NetworkInterfaces:

- DeviceIndex: "0"

AssociatePublicIpAddress: true

SubnetId: !Ref PublicSubnet

GroupSet:

- !Ref MySecurityGroup

**Note:** When using `NetworkInterfaces`, `SubnetId` and `SecurityGroupIds` must be moved inside the block — they cannot coexist at the top level.

---

## Key Concepts Learned Today

| Concept                                                     | Summary                                                                                                                                                                                       |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CloudFormation Conditions                                   | Boolean expressions defined in the <code>Conditions</code> block, evaluated at deploy time. Resources and Outputs reference them via <code>Condition:</code> key to be conditionally created. |
| <code>!Or</code> / <code>!Equals</code> intrinsic functions | Used to build compound condition logic — e.g. launch Linux if OSChoice is "Linux" OR "Both"                                                                                                   |

|                                                                             |                                                                                                                                     |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>MapPublicIpOnLaunch</code> vs <code>AssociatePublicIpAddresses</code> | Subnet-level flag is the correct way to assign public IPs; instance-level property only works inside <code>NetworkInterfaces</code> |
| <code>DeletionPolicy: Snapshot</code>                                       | Protects RDS data by taking a final snapshot before the resource is deleted                                                         |
| <code>CreationPolicy + cfn-signal</code>                                    | Gates CloudFormation stack completion until the instance signals success — prevents race conditions                                 |
| Free-tier RDS limits                                                        | Automated backups ( <code>BackupRetentionPeriod &gt; 0</code> ) are not available on free-tier accounts                             |
| Windows UserData in CFT                                                     | Must be wrapped in <code>&lt;powershell&gt;...&lt;/powershell&gt;</code> tags inside <code>Fn::Base64</code>                        |
| <code>NoEcho: true</code> on parameters                                     | Masks sensitive values (like DB passwords) in the CloudFormation console                                                            |

---

## Commands Used

# Deploy a CloudFormation stack

```
aws cloudformation deploy \
  --template-file infrastructure.yaml \
  --stack-name my-stack \
  --parameter-overrides \
    KeyName=my-key-pair \
    DBEngine=mysql \
    DBName=appdb \
    MasterUsername=dbadmin \
    MasterUserPassword=MySecurePass123! \
    AllowedSSHRDPCIDR=<YOUR_IP>/32
```

# Deploy multi-OS stack (Linux only)

```
aws cloudformation deploy \
  --template-file multi-os-infrastructure.yaml \
  --stack-name webservers \
  --parameter-overrides OSChoice=Linux KeyName=my-key
```

# Deploy multi-OS stack (Both)

```
aws cloudformation deploy \
```

```
--template-file multi-os-infrastructure.yaml \  
--stack-name webservers \  
--parameter-overrides OSChoice=Both KeyName=my-key
```

```
# Delete a stack  
aws cloudformation delete-stack --stack-name my-stack
```

```
# SSH into Linux EC2  
ssh -i my-key.pem ec2-user@<PUBLIC_IP>
```

```
# Connect to MySQL RDS from EC2  
mysql -h <RDS_ENDPOINT> -P 3306 -u dbadmin -p appdb
```

```
# Connect to PostgreSQL RDS from EC2  
psql -h <RDS_ENDPOINT> -p 5432 -U dbadmin -d appdb
```

---

## Summary

Today was focused entirely on **Infrastructure as Code using AWS CloudFormation**. Built two production-grade YAML templates covering networking, compute, and database layers. Learned how CloudFormation Conditions enable dynamic, reusable templates and debugged two real validation errors — one related to free-tier RDS limits and one related to an invalid EC2 property placement. Both errors were understood at the root cause level and fixed with the correct CFT approach.

---